MiniOS for a Study Room

Team member	KFUPM ID
Yasmeen Alshehri	202271660
Fatimah Alshehab	202278660
Raneem Alshahrani	202277080
Sarah Almas	202261060

1 Brief recap of the project idea

MiniOS is an educational C11 terminal application that simulates operating-system services through study-room activities. Users create study tasks, observe round-robin CPU scheduling and task states, manage limited task-owned memory, save revision notes, and complete simulated study sessions. The agreed scope uses one virtual CPU, up to 20 retained task records, a configurable quantum, a 65,536-byte payload budget, and virtual ticks. It runs on an existing operating system and makes scheduling, resource use, and persistence visible.

2 What has been implemented so far

Current stage: an integrated core prototype. Estimated overall project completion is 70%, using seven completed implementation work packages out of ten equally weighted project packages. The remaining three cover team acceptance checks, final report evidence, and peer review/demonstration. This is a planning estimate for the whole project; all essential core services already have working code.

Implemented area	Code written	Purpose and current behavior
Command interface	main.c, shell.c	Launch options, complete-line input, strict argument validation, and safe command routing.
Task management and lifecycle	core.c/core.h, tasks.c	Shared records, PID lookup, task creation, legal block/wake, reset, consistency checks, and shutdown.
Scheduling and metrics	scheduler.c	Circular FIFO round robin, step/run commands, virtual time, READY waiting, and final metrics.
Memory management	memory.c	PID ownership, payload budget, allocation-failure rollback, and safe release on finish/reset/exit.
Persistent notes	notes.c	Create, read, append, and delete notes; report detected file errors.
Study sessions	session.c	Represent session duration as scheduled work and announce completion once.
Logs and exports	logger.c, scheduler.c	Virtual-timestamped events, dispatch trace, completed-task CSV, and detected I/O-failure accounting.

3 System architecture

main.c initializes shared state and output files through core.c, then starts shell.c. The shell validates commands and calls task, scheduler, note, and session services. Modules share the App model from core.h. The memory registry owns task payloads, and the logger records outcomes.

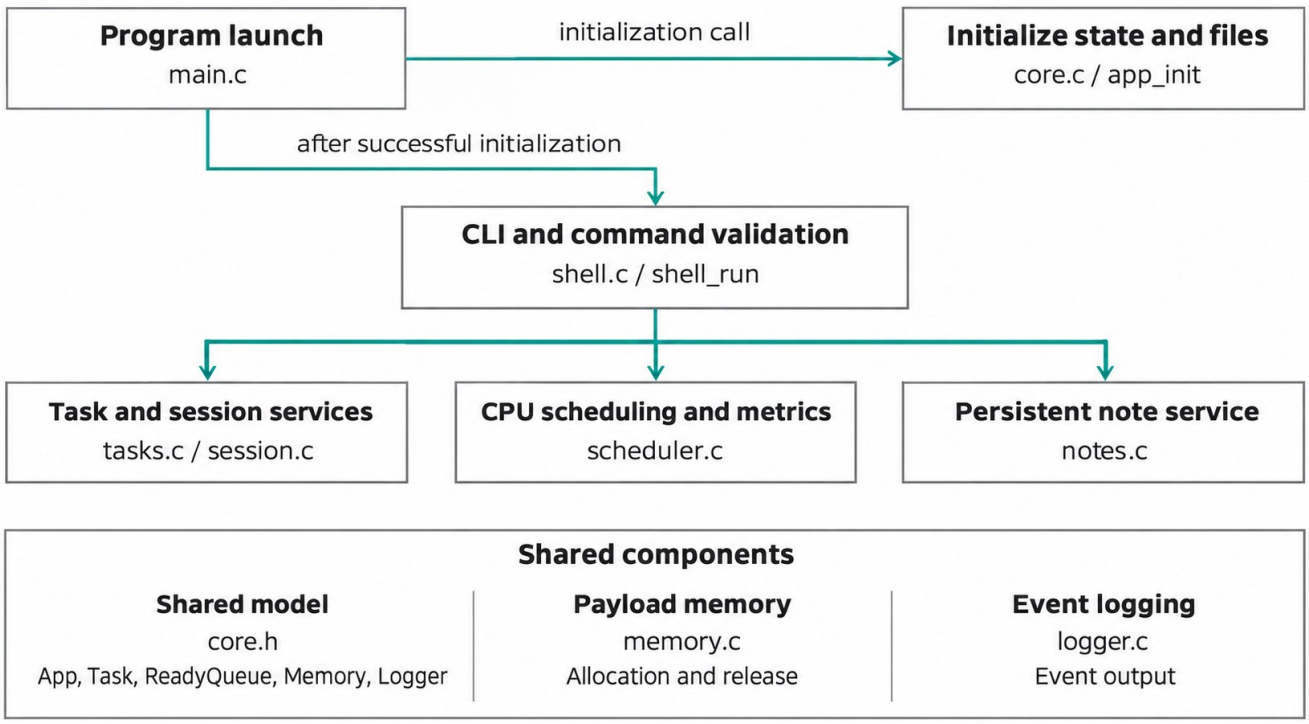


Figure 1. Current startup and command-routing architecture. The lower band identifies shared data and services.

Updates from the proposal: core.c/core.h now explicitly centralize structures and lifecycle functions. Circular ready-queue operations are implemented inside scheduler.c. Events are written directly to events.log rather than retained in a separate Event structure. The test/evidence scripts remain separate from the C application.

A successful create commits a READY task only after allocation and enqueue succeed. Dispatch changes READY to RUNNING; unfinished work returns to READY, while completion changes it to FINISHED and frees its payload. Manual block/wake moves tasks between READY and WAITING. A slice executes min(quantum, remaining); only other READY tasks accumulate waiting ticks. Notes use synchronous host-file I/O, and commands execute sequentially between slices.

4 Screenshots of the working system

The figures below are rendered console-output captures from actual executions of the C program.

```
raneem@raneem-VirtualBox:~/Downloads/Lab-Project-Phase-2-Partial-Implementation-main$ make run
mkdir -p data/notes
./minios
MiniOS for a Study Room | C11 Phase 2 prototype
One simulated CPU | quantum=2 | payload budget=65536 bytes
Type help for commands. Notes persist; task state starts fresh on restart.
minios> help
Commands (names use letters/digits/_/-; time is in virtual ticks):
  task create NAME BURST BYTES   | tasks
  session start NAME TICKS BYTES | block PID | wake PID
  quantum N | step | run | metrics | export | memory
  note create NAME TEXT | note append NAME TEXT
  note read NAME | note delete NAME
  check | reset | debug fail-next-allocation | help | quit
minios> task create Study 6 1024
Created P1 Study: READY, burst=6, payload=1024 bytes
minios> task create Review 4 512
Created P2 Review: READY, burst=4, payload=512 bytes
minios> session start Practice 3 256
Created P3 Practice: READY, burst=3, payload=256 bytes
minios> tasks
Tasks | tick=0 quantum=2 records=3/20
PID NAME                 STATE     BURST LEFT ARRIVAL BYTES
1   Study                READY     6     6    0       1024
2   Review               READY     4     4    0       512
3   Practice             READY     3     3    0       256
Ready queue: P1 P2 P3
minios>
```

Figure 2 MiniOS startup, command help, and creation of three tasks in the ready queue.

```
3   Practice             READY     3     3    0       256
Ready queue: P1 P2 P3
minios> memory
Memory: used=1792/65536 bytes free=63744 allocations=3
minios> block 2
OK: block 2
minios> tasks
Tasks | tick=0 quantum=2 records=3/20
PID NAME                 STATE     BURST LEFT ARRIVAL BYTES
1   Study                READY     6     6    0       1024
2   Review               WAITING   4     4    0       512
3   Practice             READY     3     3    0       256
Ready queue: P1 P3
minios> step
[0,2) P1 Study        RUNNING -> READY    remaining=4
minios> wake 2
OK: wake 2
minios> quantum 2
OK: quantum 2
minios> run
[2,4) P3 Practice     RUNNING -> READY    remaining=1
[4,6) P1 Study        RUNNING -> READY    remaining=2
[6,8) P2 Review       RUNNING -> READY    remaining=2
[8,9) P3 Practice     RUNNING -> FINISHED remaining=0
SESSION COMPLETE: P3 Practice at tick 9
[9,11) P1 Study        RUNNING -> FINISHED remaining=0
[11,13) P2 Review       RUNNING -> FINISHED remaining=0
Run paused: ready queue empty; WAITING tasks need wake.
minios>
```

Figure 3. Task blocking and waking, followed by round-robin execution with quantum 2; all tasks finish by tick 13.

```
Run paused: ready queue empty; WAITING tasks need wake.
minios> tasks
Tasks | tick=13 quantum=2 records=3/20
PID NAME                 STATE     BURST LEFT ARRIVAL BYTES
1   Study                FINISHED  6     0    0       1024
2   Review               FINISHED  4     0    0       512
3   Practice             FINISHED  3     0    0       256
Ready queue: (empty)
minios> metrics
PID NAME             ARR BURST FIRST DONE TURN WAIT RESP
1   Study            0   6     0     11   11   5    0
2   Review           0   4     6     13   13   7    6
3   Practice         0   3     2     9    9    6    2
Means (3 finished): turnaround=11.00 waiting=6.00 response=2.67 ticks
minios> memory
Memory: used=0/65536 bytes free=65536 allocations=0
minios> check
CHECK PASS: queue, task states, and allocation ownership are consistent.
minios> export
OK: metrics.csv exported (finished tasks only).
minios> note create demo_os_2026 Finish OS project
OK: note create demo_os_2026
minios> note append demo_os_2026 Test all commands
OK: note append demo_os_2026
minios> note read demo_os_2026
Finish OS project
Test all commands
OK: note read demo_os_2026
minios>
```

Figure 4. Task blocking and waking, followed by round-robin execution with quantum 2; all tasks finish by tick 13.

```
minios> note delete demo_os_2026
OK: note delete demo_os_2026
minios> reset
Reset: tasks cleared, clock=0, memory=0; notes preserved; quantum retained.
minios> debug fail-next-allocation
Debug hook armed: next valid allocation will fail.
minios> task create FailureTest 2 128
ERROR: Forced allocation failure; no task was created.
minios> memory
Memory: used=0/65536 bytes free=65536 allocations=0
minios> check
CHECK PASS: queue, task states, and allocation ownership are consistent.
minios> quit
Shutdown: used_bytes=0 allocations=0
raneem@raneem-VirtualBox:~/Downloads/Lab-Project-Phase-2-Partial-Implementation-main$
```

Figure 5. Note deletion, system reset, safe handling of a forced allocation failure, and clean shutdown with zero outstanding allocations.

5 Challenges faced

Challenge	Resolution or current limitation
Consistent task states and queue	Unique FIFO entries, legal block/wake transitions, and checks after commands keep the shared state consistent.
Failed allocations and cleanup	Allocate before committing a record/PID, keep one ownership registry, and test completion/reset/exit cleanup.
Waiting metrics during blocking	Accumulate READY time directly so WAITING intervals are excluded.
Invalid input and buffered file errors	Discard oversized command tails; validate complete input; check file writes and closes before reporting success.
Full leak-tool verification	LeakSanitizer cannot inspect /proc/PID/task here. The diagnostic is retained; full checking remains pending on a compatible host.

6 Plan for completing the remaining work

Remaining action	Completion evidence
Run on the agreed team computer; repeat acceptance and sanitizer checks, including full leak checking.	Recorded local results and resolved findings.
Cross-review module interfaces, ownership, state transitions, and error handling.	Reviewed code and any required corrections.
Capture local terminal screenshots and replace the rendered console captures in Section 4; finalize this report.	Working-system screenshots and a reviewed report.
Confirm team details and actual contributions; check the code package and complete the group submission.	Final report and organized C source.
Rehearse the three proposal scenarios and finalize the project demonstration.	Team demonstration and contribution record.